

AIPS Data Report

1. Objective

This report documents the construction of the dataset used in the AIPS project, whose goal is to integrate environmental variables and fishing activity data in the Gulf of Trieste.

The final dataset combines:

- Copernicus Marine environmental variables;
- Global Fishing Watch AIS vessel activity;
- Global Fishing Watch SAR vessel detections;
- calendar-based features such as holidays, weekends and fishing block periods.

The final output is a spatio-temporal dataset indexed by:

```
date, latitude, longitude
```

and saved as:

```
data/processed/cpr_gfw_2024.parquet
```

The pipeline is organized in three main steps:

```
python scripts/01_get_data.py
python scripts/02_clean_data.py
python scripts/03_build_dataset.py
```

2. Project structure

The relevant project structure is:

```
AIPS/
|
├─ config/
|   └─ config.yaml
|
└─ data/
```

```
|   |   | raw/
|   |   | interim/
|   |   | processed/
|   |
|   | notebooks/
|   |
|   | scripts/
|   |   | 01_get_data.py
|   |   | 02_clean_data.py
|   |   | 03_build_dataset.py
|   |   | check_*.py
|   |
|   | src/
|   |   | data/
|   |   |   | config.py
|   |   |   | paths.py
|   |   |   | geo.py
|   |   |   | copernicus.py
|   |   |   | gfw_ais.py
|   |   |   | gfw_sar.py
|   |   |   | build_dataset.py
|   |
|   | requirements.txt
```

The notebooks are treated as prototypes and exploratory material, while the reproducible data pipeline is now implemented through `src/` modules and `scripts/`.

3. Configuration

The pipeline is controlled by:

```
config/config.yaml
```

The main parameters are:

```
year: 2024

area:
  minimum_longitude: 13.141159647873394
  maximum_longitude: 13.8238
```

```
minimum_latitude: 45.4698
maximum_latitude: 45.8078
```

The geographical domain corresponds to the Gulf of Trieste.

The current Copernicus products are:

```
copernicus:
  products:
    - cmems_mod_med_phy-temp_my_4.2km_P1D-m
    - cmems_mod_med_bgc-plankton_my_4.2km_P1D-m
```

These products replaced the previous operational `anfc` products:

```
cmems_mod_med_phy-tem_anfc_4.2km_P1D-m
cmems_mod_med_bgc-pft_anfc_4.2km_P1D-m
```

The replacement was made because the `anfc` products did not provide a stable full-year historical coverage when re-downloaded. In particular, the 2024 re-download produced incomplete temporal ranges:

```
PHY/thetao: 2024-03-30 → 2024-12-31
BGC/chl:    2024-02-26 → 2024-12-31
```

The multiyear products instead provide the full 2024 period:

```
PHY/thetao: 2024-01-01 → 2024-12-31
BGC/chl:    2024-01-01 → 2024-12-31
```

This makes the data source more suitable for a reproducible historical analysis.

4. Step 1 — Raw data download

The first script is:

```
python scripts/01_get_data.py
```

It downloads and stores raw data from three sources:

1. Copernicus Marine
2. Global Fishing Watch AIS
3. Global Fishing Watch SAR

The raw files are saved under:

```
data/raw/
```

4.1 Copernicus Marine data

The Copernicus data are downloaded through the `copernicusmarine` Python package.

The two selected products are:

```
cmems_mod_med_phy-temp_my_4.2km_P1D-m  
cmems_mod_med_bgc-plankton_my_4.2km_P1D-m
```

The physical product provides:

```
thetao
```

which is the sea water potential temperature.

The biogeochemical product provides:

```
chl  
phyc
```

Only `chl` is currently used in the final dataset.

The raw files generated are:

```
data/raw/copernicus/cmems_mod_med_phy-temp_my_4.2km_P1D-m_2024.parquet  
data/raw/copernicus/cmems_mod_med_bgc-plankton_my_4.2km_P1D-m_2024.parquet
```

The verified raw shapes are:

```
PHY raw shape: (27084, 6)
BGC raw shape: (27084, 6)
```

The verified temporal coverage is:

```
PHY time min: 2024-01-01
PHY time max: 2024-12-31

BGC time min: 2024-01-01
BGC time max: 2024-12-31
```

The columns are:

```
PHY columns:
time, latitude, longitude, depth, bottomT, thetao

BGC columns:
time, depth, latitude, longitude, chl, phyc
```

During the raw conversion, each NetCDF file is opened with `xarray`, converted to a pandas DataFrame, filtered for non-missing marine values, and saved as Parquet.

4.2 Global Fishing Watch AIS data

The AIS data are downloaded from Global Fishing Watch using the GFW API client.

The configured GFW AIS product is:

```
public-global-fishing-effort:latest
```

The main settings are:

```
spatial_resolution: HIGH
temporal_resolution: DAILY
group_by: VESSEL_ID
```

The raw AIS file is saved as:

```
data/raw/gfw_ais/ais_high_daily_ts_2024.parquet
```

The verified raw shape is:

```
AIS raw shape: (8248, 20)
```

The verified date range is:

```
date min: 2024-01-02  
date max: 2024-12-30
```

The raw AIS columns are:

```
date  
detections  
flag  
gear_type  
hours  
vessel_ids  
vessel_id  
vessel_type  
entry_timestamp  
exit_timestamp  
first_transmission_date  
last_transmission_date  
imo  
mmsi  
call_sign  
dataset  
report_dataset  
ship_name  
lat  
lon
```

The absence of 2024-01-01 and 2024-12-31 in the raw AIS file is not necessarily an issue, since the API returns records only where vessel activity is present in the selected spatial domain.

4.3 Global Fishing Watch SAR data

The SAR data are queried from BigQuery using a Google Cloud service account stored in:

```
secrets/gcp_bigquery_key.json
```

The BigQuery table is:

```
global-fishing-watch.pipe_sentinel2_v1_published.detect_scene_match_pipe_v3
```

The raw SAR file is saved as:

```
data/raw/gfw_sar/s2_vessel_detections_2024.parquet
```

The selected SAR fields are:

```
ssvid  
detect_timestamp  
detect_lat  
detect_lon  
speed_kn_inferred  
heading_deg_inferred  
length_m_inferred  
presence_score  
matching_score  
cloud_score  
likely_infrastructure  
date
```

A technical correction was applied before saving the SAR raw file: the BigQuery `date` column was converted to string format `YYYY-MM-DD`. This avoids Parquet reading errors caused by the BigQuery-specific `dbdate` dtype.

5. Step 2 — Data cleaning

The second script is:

```
python scripts/02_clean_data.py
```

It reads raw data from:

```
data/raw/
```

and writes cleaned intermediate files to:

```
data/interim/
```

5.1 Copernicus cleaning

The Copernicus cleaning is implemented in:

```
src/data/copernicus.py
```

The cleaning procedure is applied separately to the physical and biogeochemical products.

The main operations are:

1. read raw Parquet file;
2. check that only one depth level is present;
3. drop the `depth` column;
4. rename `time` to `date`;
5. remove points outside the selected marine domain using the Italian marine boundary;
6. save the cleaned output to `data/interim/copernicus/`.

The cleaned files are:

```
data/interim/copernicus/cleaned_cmems_mod_med_phy-temp_my_4.2km_P1D-  
m_2024.parquet  
data/interim/copernicus/cleaned_cmems_mod_med_bgc-plankton_my_4.2km_P1D-  
m_2024.parquet
```

The geographical filtering uses:

```
data/raw/italy_marine_bounds.csv
```

and the function:

```
filter_above_boundary()
```

defined in:

```
src/data/geo.py
```

5.2 AIS cleaning

The AIS cleaning is implemented in:

```
src/data/gfw_ais.py
```

The main operations are:

1. read the raw AIS Parquet file;
2. drop metadata columns not used in the final dataset;
3. rename:

```
lat → latitude  
lon → longitude
```

1. select the relevant columns:

```
date  
latitude  
longitude  
gear_type  
mmsi  
ship_name  
hours  
flag
```

1. keep only Italian vessels:

```
df = df[df["flag"] == "ITA"]
```

1. filter points using the Italian marine boundary;
2. save the cleaned output.

The cleaned AIS file is:

```
data/interim/gfw_ais/cleaned_ais_high_daily_ts_2024.parquet
```

No transformation is applied to the `gear_type` variable. In particular, the original notebooks did not contain any rule such as:

```
FISHING → TRAWLERS
```

The original pipeline simply counted vessels by their existing `gear_type`.

5.3 SAR cleaning

The SAR cleaning is implemented in:

```
src/data/gfw_sar.py
```

The main operations are:

1. read the raw SAR Parquet file;
2. drop `detect_timestamp`;
3. rename:

```
detect_lat → latitude  
detect_lon → longitude
```

1. apply quality filters;
2. remove filtering metadata columns;
3. filter points using the Italian marine boundary;
4. save the cleaned output.

The SAR filters are:

```
speed_kn_inferred < 9  
length_m_inferred < 25  
presence_score > 0.5  
cloud_score < 0.5  
likely_infrastructure == False
```

The cleaned SAR file is:

```
data/interim/gfw_sar/cleaned_s2_vessel_detections_2024.parquet
```

6. Step 3 — Final dataset construction

The third script is:

```
python scripts/03_build_dataset.py
```

The build logic is implemented in:

```
src/data/build_dataset.py
```

The script reads:

```
data/interim/copernicus/  
data/interim/gfw_ais/  
data/interim/gfw_sar/
```

and writes:

```
data/processed/cpr_gfw_2024.parquet
```

6.1 Date standardization

All datasets are standardized by converting their date column to pandas datetime format:

```
pd.to_datetime(df["date"])
```

This ensures that Copernicus, AIS and SAR can be merged on a common temporal index.

6.2 Copernicus grid construction

The final dataset is built on the Copernicus spatial-temporal grid.

The base keys are:

```
date
latitude
longitude
```

The base DataFrame is created from the unique Copernicus BGC coordinates and dates:

```
keys = ["date", "latitude", "longitude"]

df = cpr_bgc.drop_duplicates(subset=keys)[keys].copy()
```

Then the environmental variables are merged:

```
chl
thetao
```

The resulting base grid contains one row for each available combination of day and Copernicus grid cell.

6.3 Projection of vessel detections onto Copernicus grid

AIS and SAR vessel positions do not necessarily lie exactly on the Copernicus grid.

Therefore, vessel coordinates are projected to the nearest Copernicus grid point using:

```
project_to_nearest()
```

defined in:

```
src/data/geo.py
```

The procedure is:

1. extract unique vessel coordinates;
2. extract unique Copernicus grid coordinates;
3. find the nearest grid point for each vessel coordinate;
4. replace the original vessel coordinates with the nearest Copernicus grid coordinates.

This allows AIS, SAR and Copernicus data to be aggregated on the same spatial grid.

6.4 SAR aggregation

After projection, SAR detections are counted by:

```
date
latitude
longitude
```

The resulting variable is:

```
sar_vessels_count
```

Missing counts are filled with zero.

6.5 AIS aggregation

After projection, AIS detections are counted by:

```
date
latitude
longitude
```

The total AIS count is stored in:

```
ais_vessels_count
```

In addition, AIS vessels are counted by gear type:

```
DREDGE_FISHING
FISHING
OTHER_PURSE_SEINES
POTS_AND_TRAPS
TRAWLERS
```

The notebook and the new pipeline both use the existing `gear_type` labels without forcing or reclassifying them.

Missing count values are filled with zero.

6.6 Calendar features

The final dataset includes calendar-based variables:

```
is_holiday  
holiday_name  
is_weekend  
fishing_block
```

Italian public holidays are added using the `holidays` Python package.

The weekend flag is computed from the day of the week:

```
df["is_weekend"] = df["date"].dt.dayofweek >= 5
```

For 2024, the fishing block is defined as:

```
2024-07-31 → 2024-09-13
```

7. Final dataset

The final processed file is:

```
data/processed/cpr_gfw_2024.parquet
```

The final dataset shape is:

```
(17934, 16)
```

The temporal coverage is:

```
2024-01-01 → 2024-12-31
```

The final columns are:

```
date
is_holiday
holiday_name
is_weekend
fishing_block
latitude
longitude
chl
thetao
sar_vessels_count
ais_vessels_count
DREDGE_FISHING
FISHING
OTHER_PURSE_SEINES
POTS_AND_TRAPS
TRAWLERS
```

The final aggregate checks are:

```
sar_vessels_count sum: 2604
ais_vessels_count sum: 4039
```

The environmental variables are derived from the new Copernicus multiyear products:

```
thetao
chl
```

8. Validation against notebook output

The final dataset produced by the new pipeline was compared with the previous dataset produced from the notebooks:

```
data/processed/cpr_gfw_2024.parquet
data/processed/cpr_gfw_2024_from_notebook.parquet
```

The comparison showed:

```
SAR count:
new = 2604
```

```
old = 2604
difference = 0
```

The AIS total count was almost identical:

```
new = 4039
old = 4035
difference = 4
```

The spatial grid was also consistent:

```
latitude difference = 0
longitude difference ≈ 0
```

Small differences were observed in some AIS gear-type columns:

```
FISHING: +45
TRAWLERS: -41
DREDGE_FISHING: -4
POTS_AND_TRAPS: +3
OTHER_PURSE_SEINES: +1
```

A check on the original notebook code confirmed that no rule was applied to force `FISHING` vessels into `TRAWLERS`. The original notebook only counted the existing `gear_type` values.

Therefore, these small differences are likely due to updated GFW metadata or small changes in the AIS data returned by the API.

Differences in `thetao` and `chl` are expected because the new pipeline now uses Copernicus multiyear/reanalysis products rather than the previous operational `anfc` products.

The validation supports the conclusion that:

- the new pipeline reproduces the original notebook logic;
- the spatial-temporal grid is consistent;
- SAR counts are identical;
- AIS counts are nearly identical;
- environmental differences are explained by the controlled change of Copernicus source products.

9. Reproducibility notes

The pipeline depends on two credential files stored locally and excluded from Git:

```
secrets/gfw_token.txt
secrets/gcp_bigquery_key.json
```

The following folders should not be committed to GitHub:

```
data/
secrets/
.venv/
__pycache__/
.ipynb_checkpoints/
```

The current recommended `.gitignore` entries are:

```
.venv/
data/
secrets/
__pycache__/
.ipynb_checkpoints/
*.nc
*.parquet
.DS_Store
```

The full reproducible workflow is:

```
python scripts/01_get_data.py
python scripts/02_clean_data.py
python scripts/03_build_dataset.py
```

After running these commands, the final dataset is available at:

```
data/processed/cpr_gfw_2024.parquet
```

10. Summary

The notebook-based workflow was successfully converted into a reproducible Python pipeline.

The main methodological improvement was the replacement of operational Copernicus `anfc` products with multiyear/reanalysis products, which provide stable full-year historical coverage.

The final dataset integrates environmental covariates and vessel activity data on a common daily Copernicus grid for the Gulf of Trieste.

The final processed dataset has:

```
shape: (17934, 16)
period: 2024-01-01 → 2024-12-31
SAR vessel count total: 2604
AIS vessel count total: 4039
```

This dataset is now suitable for downstream exploratory analysis, statistical modelling and predictive modelling of fishing activity in the Gulf of Trieste.